ELEC 475 Lab 2

By:

Josh Poplove 20334465

Mihir Gandhi 20336712

2. Architectural Specifications

All models specified in this lab received inputs of RGB images of the size 3x64x64. The CIFAR-10 images must be resized to the specified size, while the Tiny ImageNet images are already configured to that size. For CIFAR-10, the number of output classes, represented by *C,* is equal to 10. For Tiny ImageNet, the number of output classes is 200. For pooling and convolution layers, the spatial output dimension can be calculated using the equation below.

$$N_{out} = \left\lfloor \frac{N_{in} + 2P - K}{S} \right\rfloor + 1,$$

Where K is there kernel size, P is the padding size, and S is the stride. For a convolution layer with bias, it has (K^2 C_in + 1) C_out trainable parameters, and a fully connected layer with bias has (N_in + 1) N_out parameters.

2.1 Base LeNet

The Base LeNet architecture uses two 5x5 convolution layers, one with 6 channels and another with 16 channels. It also has a stride of 1 and no padding. Following the convolution layers, Tanh is applied along with 2 by 2 average pooling with stride 2. It has no normalization or dropout in the fully connected layers.

The feature dimensions proceed as follow: $3 \times 64 \times 64 \rightarrow 6 \times 60 \times 60 \rightarrow 6 \times 30 \times 30 \rightarrow 16 \times 26 \times 26 \rightarrow 16 \times 13 \times 13$

After flattening, there are 2704 features produced. The classifier goes 2704 → 120 →84 →C

Tanh restricts activations to values between -1 and 1, while average pooling replaces each 2x2 region with its mean. The dense layers combine the extracted features into class scores. Most parameters are in FC1, which connects all 2,704 features to 120 neurons.

2.2 Modernized LeNet

Modernized LeNet uses the same 6 and 16 channel convolutions as base LeNet, but uses ReLU instead. It also uses a convolution passing of 2, and overlapping 3x3 max pooling with stride of 2.

The feature dimensions proceed as follows: $3 \times 64 \times 64 \rightarrow 6 \times 64 \times 64 \rightarrow 6 \times 31 \times 31 \rightarrow 16 \times 31 \times 31 \rightarrow 16 \times 15 \times 15$

The classifier goes 3600 → 120 →84 →C. The larger flattened input increases FC1 from 324,600 to 432,120 parameters.

ReLU applies f(x)=max(0,x). Max pooling selects the strongest response in each region, and its windows overlap because the stride is smaller than the kernel. Padding preserves spatial size during convolution, so the larger flattened vector results from both the padding and pooling changes. No normalization or dropout is used.

2.3 AlexNet-64

AlexNet-64 has five convolutions with channels 64, 192, 384, 384, 256. Conv1 uses kernel of 5, stride of 2, and padding of 2. Conv2 uses a kernel of 5, stride of 1, and padding of 2. Conv3 to Conv5 uses kernel of 3, stride of 1, and padding of 1. All stages use ReLU. Max pooling with kernel 3 and stride 2 follows stages 1, 2, and 5.

Stage	Conv Output	Post Pooling Output
1	64x32x32	64x15x15
2	192 x 15 x 15	192 x 7 x 7
3	384 x 7 x 7	N/A
4	384 x 7 x 7	N/A
5	256 x 7 x 7	256 x 3 x 3

The classifier with ReLU after the two hidden layers and a dropout of p=0.5 before each hidden dense layer goes 2304 → 1024 →1024 →C

Stages 3-5 preserve the 7x7 spatial resolution while combining information across channels. The last pool produces 256x3x3=2304 features. During training, dropout randomly zeros half its input activations on average and it is disabled during evaluation. The None variant has no normalization. LRN precedes ReLU in stages 1-2 in the implemented code, using size=5, alpha=0.0001, beta=0.75, and k=2. BatchNorm follows all five convolutions before ReLU, adding 2(64+192+384+384+256)=2560 trainable scale/offset parameters. Augmentation does not change the architecture.

LRN adds no trainable parameters. BatchNorm has one learned scale and offset per channel. Its running means and variances are stored as buffers rather than trainable weights.

2.4 PyTorch Zoo AlexNet

The TorchVision baseline uses five convolutions with channels 64, 192, 384, 256, 256. Conv1 uses kernel of 11, stride of 4, and padding of 2. Conv2 uses a kernel of 5, stride of 1, and padding of 2. Conv3 to Conv5 uses kernel of 3, stride of 1, and padding of 1. All stages use ReLU. Max pooling with kernel 3 and stride 2 follows stages 1, 2, and 5.

The spatial dimension progress as follows: $64 \rightarrow 15 \rightarrow 7 \rightarrow 7 \rightarrow 3 \rightarrow 3 \rightarrow 3 \rightarrow 3 \rightarrow 1$

Adaptive average pooling maps 256x1x1 to 256x6x6. The classifier is 9216→4096→4096→C, with ReLU after the hidden layers and dropout p=0.5 before them. There is no BN or LRN. The large dense layers account for most parameters. The initial stride of 4 reduces spatial resolution more aggressively than AlexNet-64's stride of 2. Adaptive pooling repeats the final 1x1 values to fit the fixed classifier input; it does not restore spatial detail. Its 4,096-unit dense layers make the classifier substantially larger than AlexNet-64's 1,024-unit layers.

2.5 Parameter Breakdown

For convolutions with bias, parameters equal $(\mathrm{K}^2\mathrm{C}_{in}+1)\mathrm{C}_{out}$. For dense layers, they equal $(\mathrm{n}_{in}+1)\mathrm{n}_{out}$. For example, LeNet Conv1 has $(5^2\times 3+1)\times 6=456$ parameters. Pooling, activations, dropout, and flattening add no trainable parameters.

Model	**Feature extractor**	**Classifier, C=10**	**Total, C=10**	**Total, C=200**
Base LeNet	2,872	335,614	338,486	354,636
Modernized LeNet	2,872	443,134	446,006	462,156
AlexNet: None/LRN	3,188,672	3,420,170	6,608,842	6,803,592
AlexNet: BN	3,191,232	3,420,170	6,611,402	6,806,152
Zoo AlexNet	2,469,696	54,575,114	57,044,810	57,823,240

Only the final dense layer changes between datasets. For example, AlexNet's 200-class output has $(1024+1)\times 200=205000$ parameters, compared with 10,250 for 10 classes. All models output raw logits.

2.6 Receptive Fields

The receptive field "r" measures the input-region extent associated with one feature position. The sampling jump "j"is the distance between adjacent feature positions in input pixels. Starting with $\mathrm{r}_0=1$ and $\mathrm{j}_0=1$, each convolution or pooling layer gives the formula below

$$\mathrm{r}_l=\mathrm{r}_{l-1}+(\mathrm{K}_l-1)\mathrm{j}_{l-1},\quad \mathrm{j}_l=\mathrm{S}_l\mathrm{j}_{l-1}$$

Model	**Successive receptive field sizes**	**Final size before the dense classifier**

Base LeNet	5, 6, 14, 16	16 x 16
Modernized LeNet	5, 7, 15, 19	19 x 19
AlexNet-64	5, 9, 25, 33, 49, 65, 81, 97	97 x 97
Zoo AlexNet	11, 19, 51, 67, 99, 131, 163, 195	195 x 195

These are geometric receptive fields at the final spatial pool. Values exceeding 64 include padded regions. The available image remains 64x64. Zoo adaptive pooling repeats the final 1x1 values without adding image information. Dense layers combine the flattened features.

3. Empirical Results and Figures

3.1 Master Ablation Summary Tables

Table 7.3a: Architectural Configuration & Computational Complexity Profile

Model Architecture	Conv Depth	Normalization & Regularization	Data Augmentation	Trainable Params (M)	FLOPs (MFLOPs)	Mean Epoch Time (s)	Inference Latency (ms)
1. Base LeNet-5	2	None	None	0.338	7.17	21.92	0.49
2. Modernized LeNet	2	None (ReLU + Overlapping MaxPool)	None	0.446	9.19	22.17	0.50
3. AlexNet (No Norm)	5	Dropout (p=0.5)	None	6.609	436.70	26.52	2.20
4. AlexNet + LRN	5	LRN + Dropout (p=0.5)	None	6.609	436.70*	28.38	1.90
5. AlexNet + BatchNorm	5	BatchNorm2d + Dropout (p=0.5)	None	6.611	437.97	27.11	1.77
6. AlexNet + BN + Aug	5	BatchNorm2d + Dropout (p=0.5)	Pure PyTorch Aug	6.611	437.97	59.53	1.77
7. PyTorch Zoo AlexNet	5	None (AdaptiveAvgPool2d)	None	57.045	188.20	28.4	2.22

Step 3: Δ = Train Acc − Val Acc

No aug: 90.31 – 78.59 = 11.72

With aug:84.64- 81.65 = 2.99

Table 7.3b: Empirical Predictive Performance Across Datasets

Model Architecture	CIFAR-10 Val Acc (%)	CIFAR-10 Val Macro F1 (%)	Tiny ImageNet Val Acc (%)	Tiny ImageNet Val Macro F1 (%)
1. Base LeNet-5	55.23	55.29	18.95	16.57

2. Modernized LeNet	66.65	66.82	26.64	24.91
3. AlexNet (No Norm)	73.52	73.48	18.03	15.13
4. AlexNet + LRN	75.14	75.26	16.50	13.69
5. AlexNet + BatchNorm	79.93	80.06	30.15	28.25
6. AlexNet + BN + Aug	82.83	82.90	31.65	30.43
7. PyTorch Zoo AlexNet	65.02	65.02	7.51	4.99

Best validation-epoch values reported for each model.

3.2 Extended Ablation Summary Table

Each row changes one factor; Δ = after − before (best validation accuracy, %). 10 epochs unless noted.

Change (from → to)	**Dataset**	**Before (%)**	**After (%)**	**Δ (pts)**
Tanh + AvgPool → ReLU + MaxPool (LeNet)	CIFAR-10	55.23	66.65	+11.42
No norm → LRN	CIFAR-10	73.52	75.14	+1.62
No norm → BatchNorm	CIFAR-10	73.52	79.93	+6.41
No norm → LRN	Tiny ImageNet	18.03	16.50	−1.53
No norm → BatchNorm	Tiny ImageNet	18.03	30.15	+12.12
No aug → aug, BN (15 epochs)	CIFAR-10	81.18	82.83	+1.65
No aug → aug, BN	Tiny ImageNet	30.15	31.65	+1.50
AlexNet-64 → Zoo AlexNet	CIFAR-10	79.93	65.02	−14.91

AlexNet-64 → Zoo AlexNet	Tiny ImageNet	30.15	7.51	−22.64

3.3 Multi-seed evaluation result

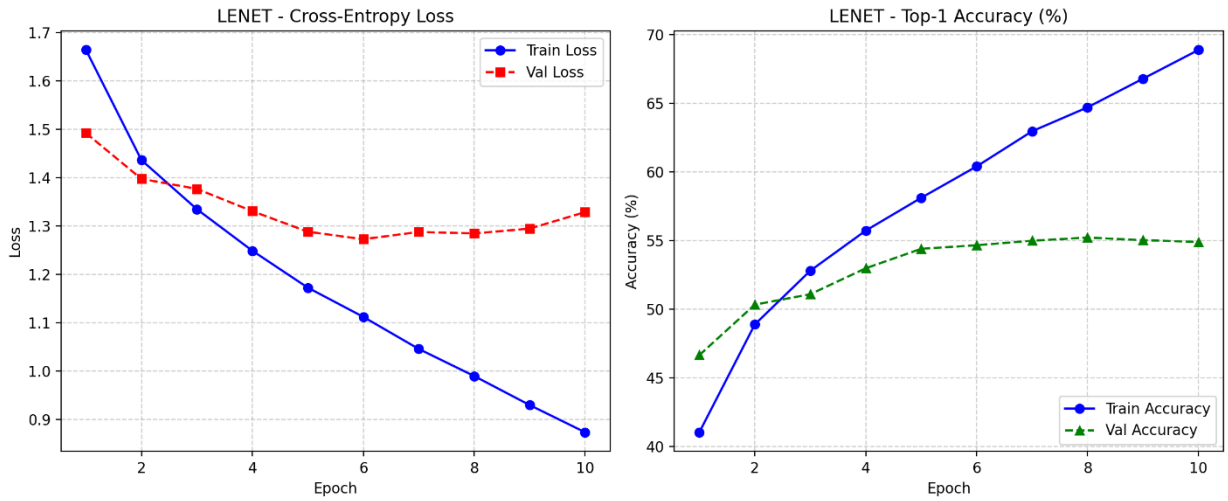
To measure run-to-run variation, AlexNet + Batchnorm was trained on CIFAR-10 with three random seeds (10 epochs, no augmentation, Tesla T4), using the global seeding in utils/seed.py. All other settings are the same as Variant C.

Seed	**Best Val Acc (%)**	**Best Val Macro F1 (%)**
42	79.93	80.06
43	78.56	78.63
44	80.19	79.99
Mean ± Std	**79.56 ± 0.88**	**79.56 ± 0.81**

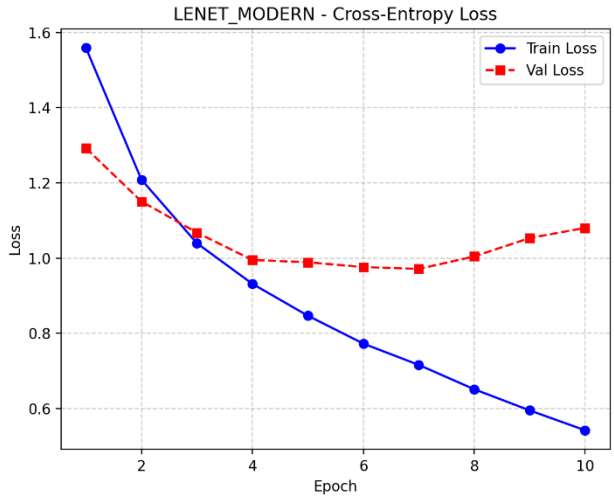
Validation accuracy varied by about 0.9 points between seeds.

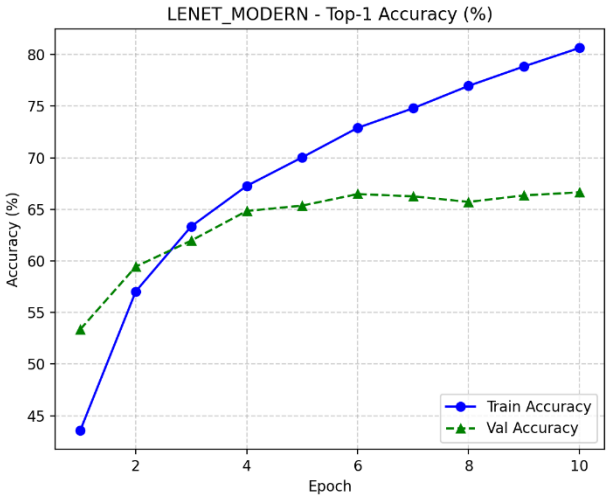
3.4 Training Curves

LeNet Regular:

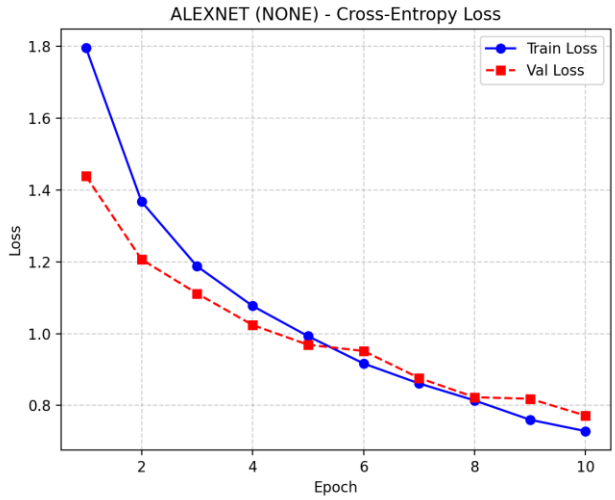


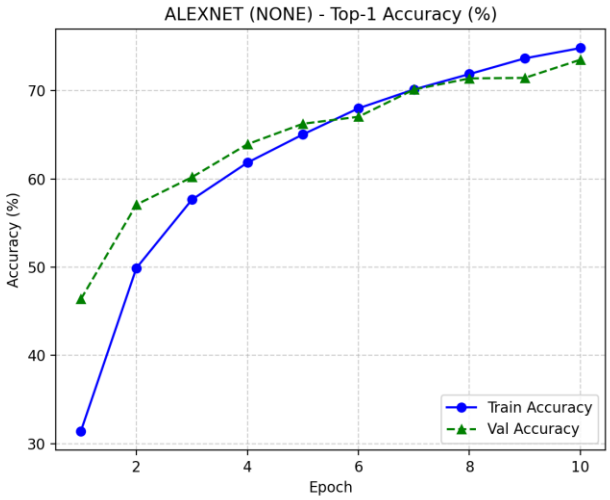
LeNet Modern:



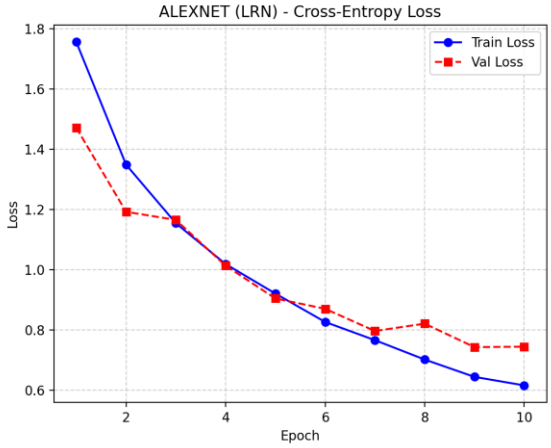


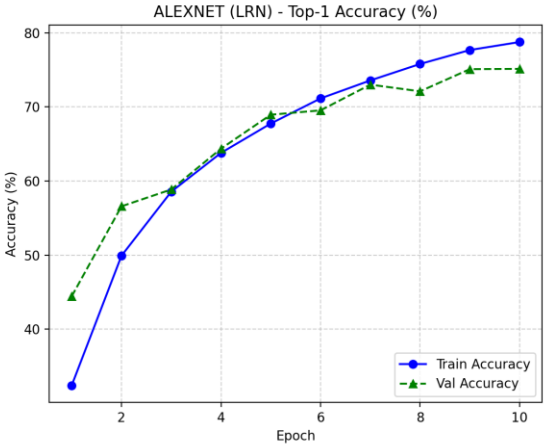
AlexNet no norm:



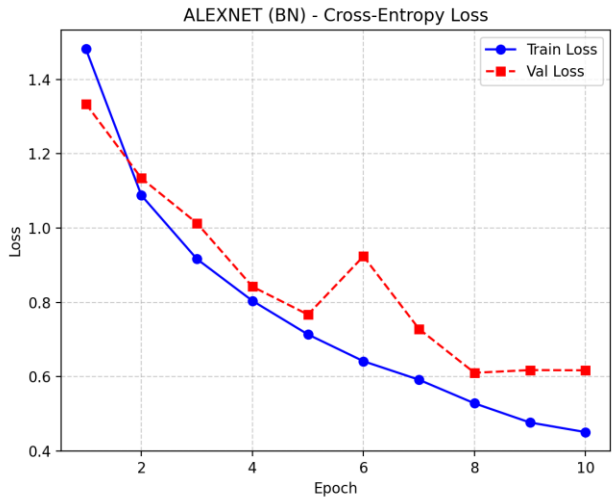


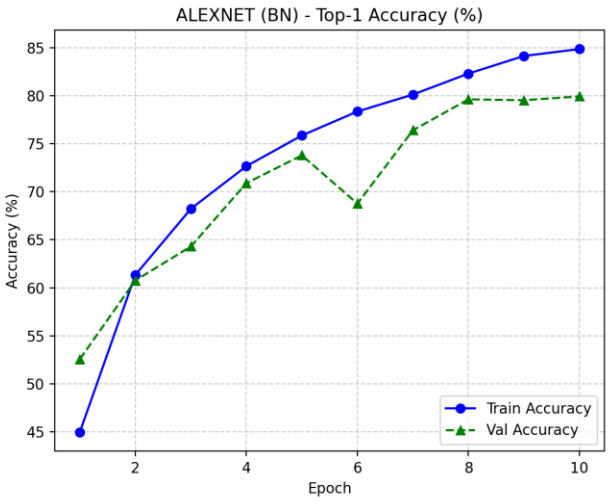
AlexNet LRN:



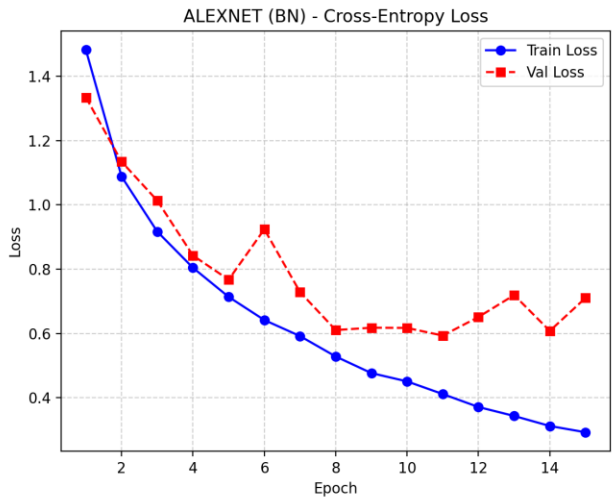


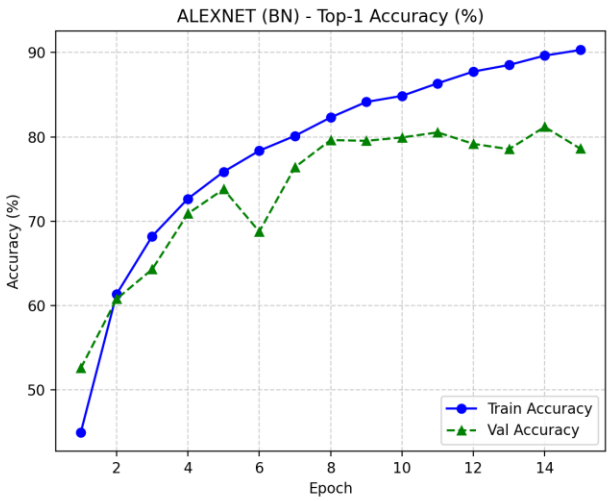
AlexNet BN:



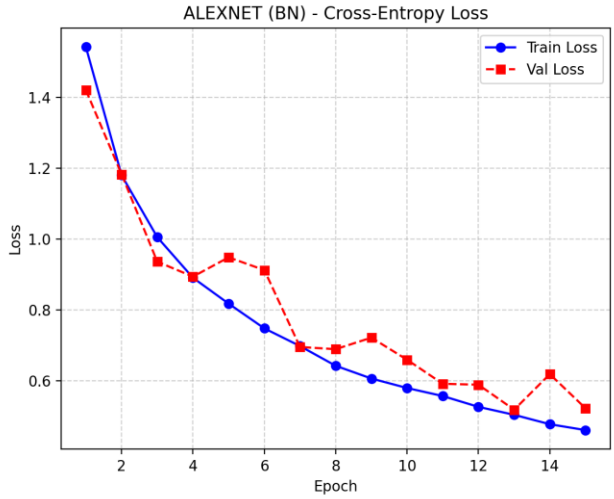


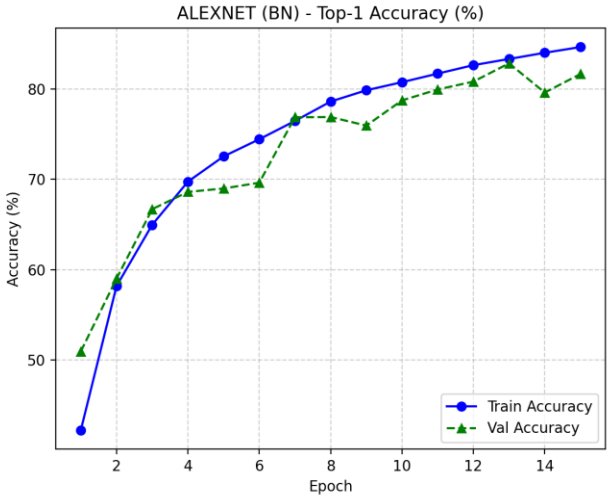
No augmentation:



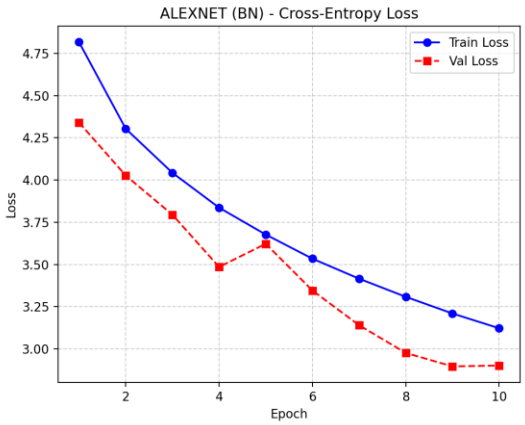


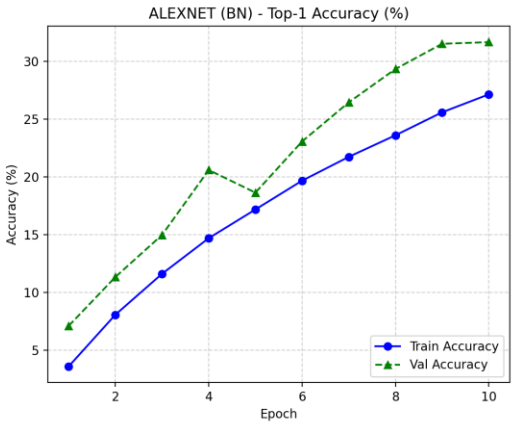
With augmentation:



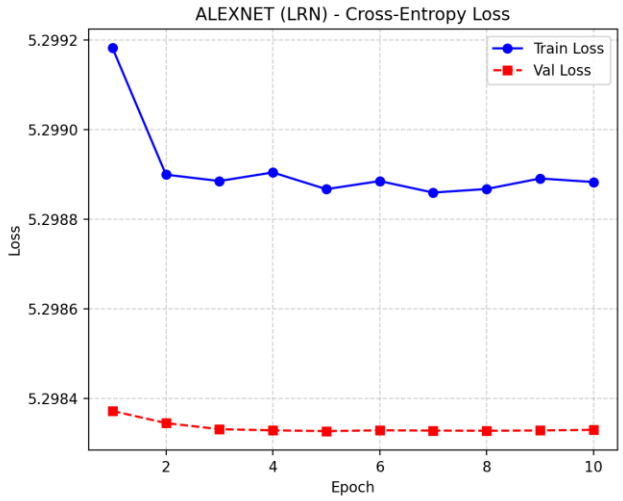


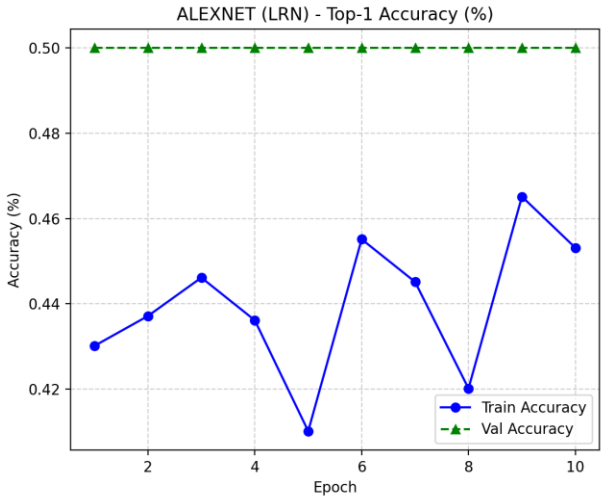
Tiny ImageNet (BN + aug)



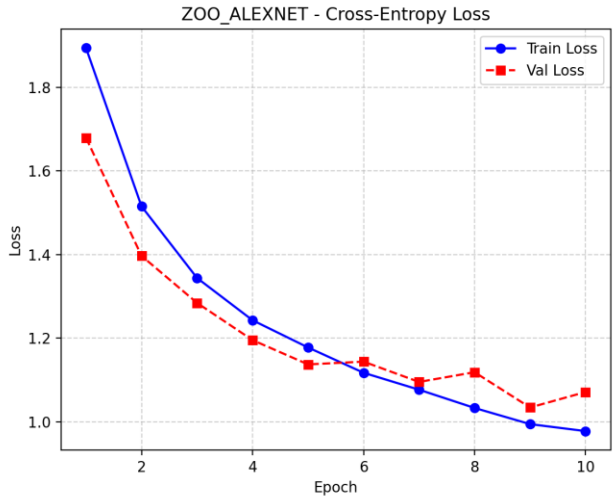


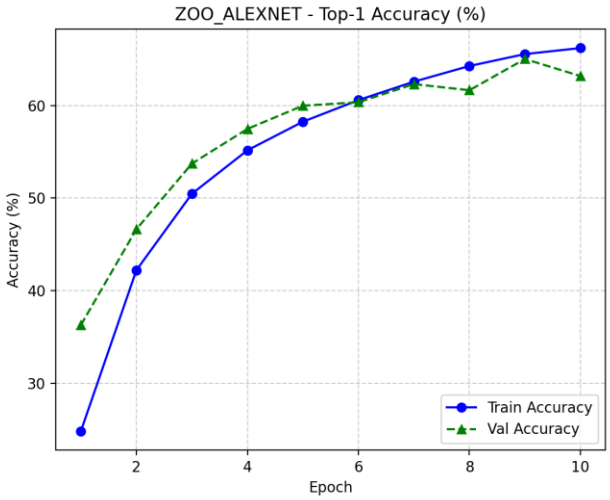
AlexNet LRN Tiny:



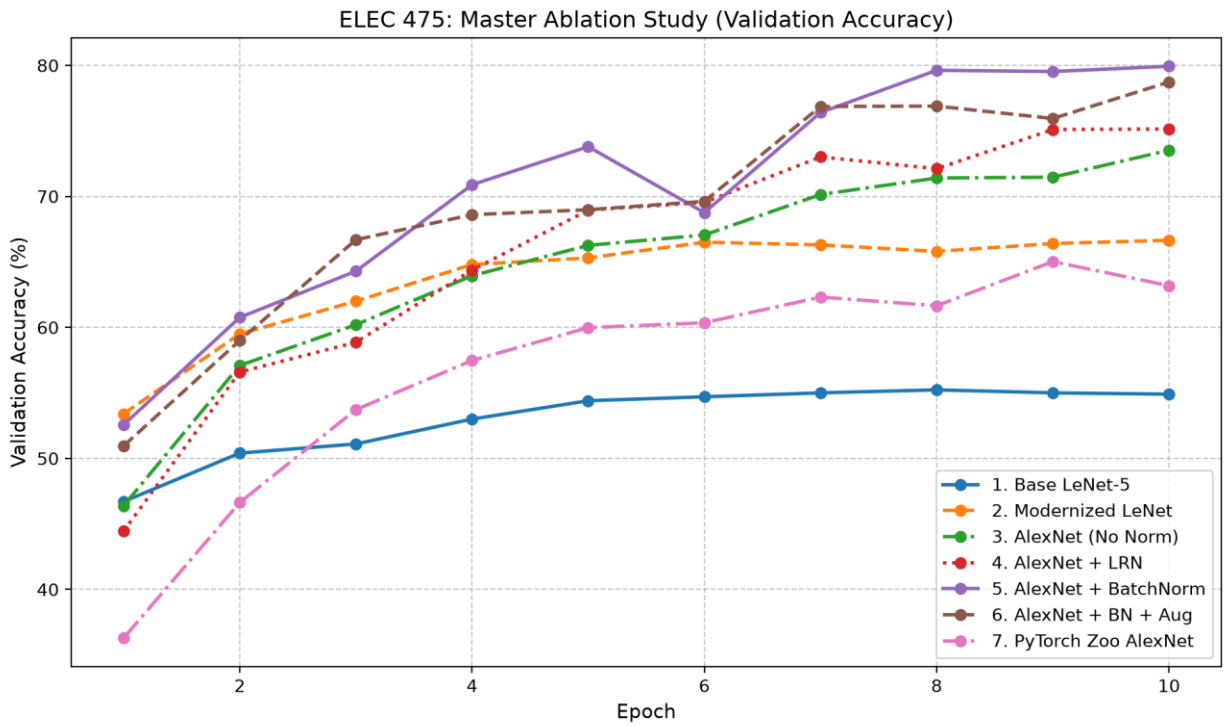


Zoo:





Ablation Summary:



3.5 “demo_transforms” Output

The output of the file *demo_transforms.py* is:

```
================================================================================
ELEC 475: Transform Paradigm & Variable Type Diagnostic
================================================================================

[1] OpenCV (cv2) Output:
    - Object Type      : <class 'numpy.ndarray'>
    - Data Type (dtype): float32
    - Shape (Layout)   : (64, 64, 3)  --> [Height, Width, Channels] (Channels-
Last)
    - Value Range      : Min = 0.0000, Max = 1.0000
    - Color Space      : Converted BGR -> RGB
    - Memory Location  : Host System RAM (CPU)

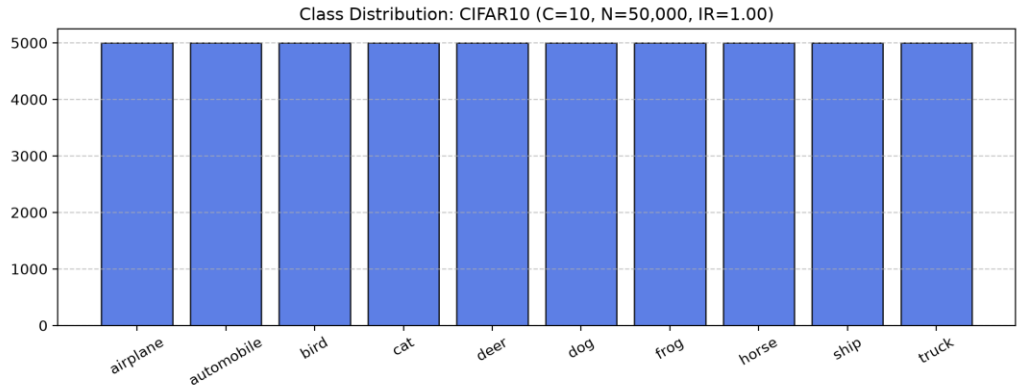
[2] NumPy Output:
    - Object Type      : <class 'numpy.ndarray'>
    - Data Type (dtype): float32
    - Shape (Layout)   : (64, 64, 3)  --> [Height, Width, Channels] (Channels-
Last)
    - Value Range      : Min = 0.0000, Max = 1.0000
    - Memory Location  : Host System RAM (CPU)
```

```
[3] PyTorch (torchvision) Tensor Output:
    - Object Type      : <class 'torch.Tensor'>
    - Data Type (dtype): torch.float32
    - Shape (Layout)   : torch.Size([3, 64, 64])  --> [Channels, Height, Width]
(Channels-First)
    - Value Range      : Min = -1.9895, Max = 2.1265 (Standardized)
    - Memory Location  : cpu (Host CPU RAM)

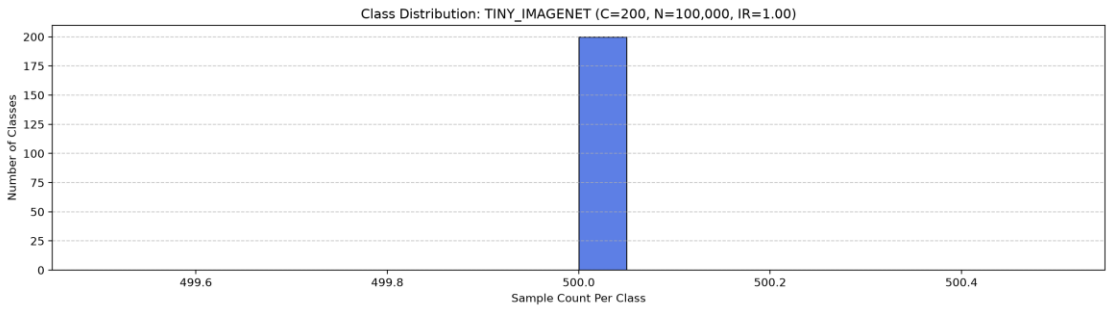
--------------------------------------------------------------------------------
Benchmark: Processing 1,000 Images (Resize + Flip + Normalize)
--------------------------------------------------------------------------------
- OpenCV (CPU)            :   16.43 ms  ( 60,863 img/s) + Host-to-Device Copy
required
- NumPy (CPU)             :   60.97 ms  ( 16,403 img/s) + Host-to-Device Copy
required
- PyTorch (CPU Tensors)   :  162.39 ms  (  6,158 img/s)
================================================================================
```

3.6 EDA Figures

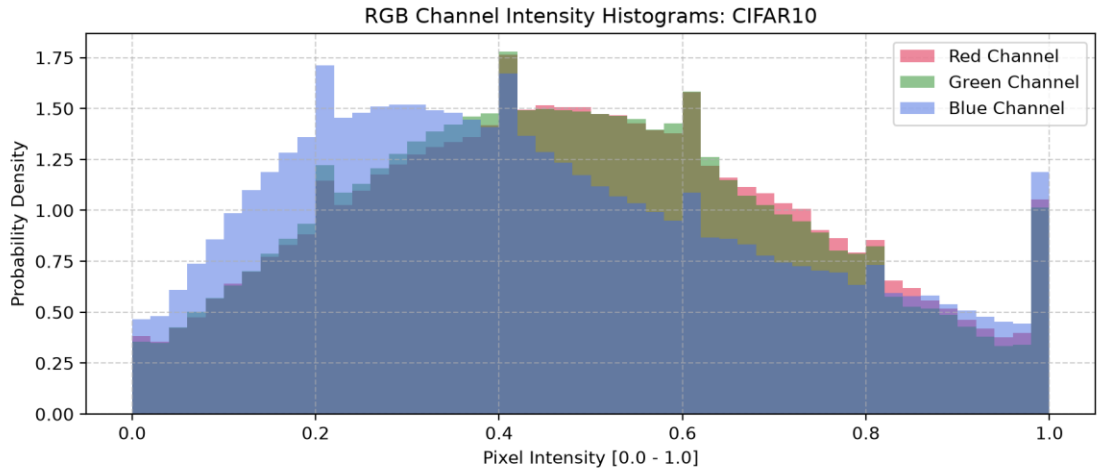
CIFAR10 class distribution:



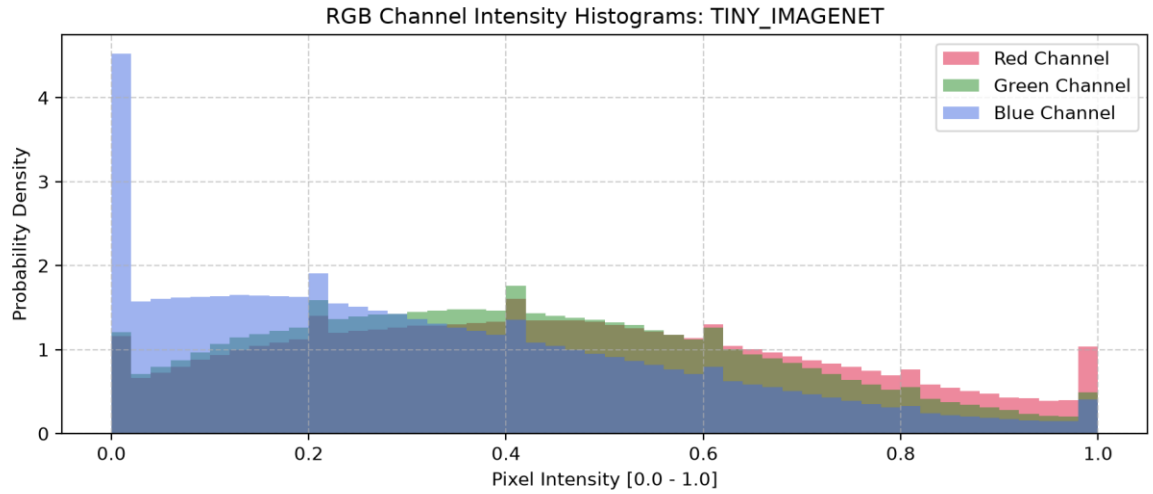
Tiny imagenet class distribution:



CIFAR10 channel histogram:



Tiny imagenet channel histograms:



3.7 compare_models Output

The output of *compare_models.py* is:

```
================================================================================
ELEC 475: Lab 2 - Comprehensive Model Complexity & FLOPs Benchmark
Device         : cuda (Tesla T4)
Dataset        : CIFAR10 (10 classes)
Input Shape    : (1, 3, 64, 64)
================================================================================
```

```
Instantiating models for profiling...
Profiling FLOPs, Multiply-Accumulates, and Latency...

| Model Architecture          | Params (M) | MACs (M) | FLOPs (M) | Latency (ms)
| Throughput (img/s) |
| ----------------------------|------------|----------|-----------|--------------
|------------------- |
| 1. LeNet-5 (Base)           | 0.338      | 3.59     | 7.17      |
0.49         | 2057.1              |
| 2. LeNet-Modern (ReLU+Pool) | 0.446      | 4.59     | 9.19      |
0.50         | 1987.1              |
| 3. AlexNet-64 (None)        | 6.609      | 218.35   | 436.70    |
2.20         | 455.3               |
| 4. AlexNet-64 (LRN)         | 6.609      | 218.35   | 436.70    |
1.90         | 527.3               |
| 5. AlexNet-64 (BatchNorm)   | 6.611      | 218.98   | 437.97    |
1.77         | 564.8               |
| 6. Torchvision Zoo AlexNet  | 57.045     | 94.10    | 188.20    |
2.22         | 450.5               |

================================================================================
Academic Notes on Complexity Metrics:
1. FLOPs vs MACs: 1 Multiply-Accumulate (MAC) corresponds to ~2 Floating Point
Operations (FLOPs).
2. Memory vs Compute: Dense Linear layers contribute the majority of parameters,
but Conv layers dominate FLOPs.
3. Reference: He et al. (CVPR 2016), Howard et al. (MobileNets 2017).
================================================================================
```

4. Technical Analysis and Discussion

4.1 Empirical Normalization Study

Performance Improvement: Yes, batch normalization gave the best result in CIFAR-10, raising best validation accuracy from 73.52% (no norm) to 75.14% (LRN) to 79.93%, and Macro F1 from 73.48% and 75.26% to 80.06%. It also learned the fastest, reaching 52.56% after the first epoch compared to 46.38% and 44.45%. BatchNorm2d normalizes each channel to 0 mean and unit variance over the mini-batch and then applies a learned scale (γ) and shift (β). Each layer therefore receives inputs in a stable range even while earlier

layers are changing, which reduces internal covariate shift. In practice, this smooths the loss landscape and makes gradient magnitudes more predictable, so the same learning rate makes more consistent progress, and keeping pre-activations centred limits the number of inactive ReLU units. LRN only rescales each activation by its neighbouring channels, with no batch statistics and no learned parameters, which explains its much smaller gain with +1.62 points.

The benefit was larger on harder dataset. On Tiny ImageNet, BN improved from 18.03% to 30.15%, which is almost twice the CIFAR-10 gain, while LRN gave no improvement with 16.50%

Computational Overhead: yes, the addition of BatchNorm2d altered the training epoch clock time and arithmetic FLOPs, but by a little bit. On the T4, mean epoch time was 26.52 s with no normalization, 28.38 s with LRN (+7%) and 27.11 s with BN (+2%). BN added only 0.3% FLOPs (436.70 to 437.97 MFLOPs). The FLOP counter (thop) reports no change for LRN because it has no rule for LRN layers. Its real cost shows up in training time instead: 28.38 s per epoch versus 26.52 s with no normalization (+7%), compared with +2% for BN. At batch size 1 on the GPU, inference latencies (2.20, 1.90 and 1.77 ms) are dominated by kernel-launch overhead and do not separate the three variants.

Convergence stability: BatchNorm had the most stable gradient progression. Training loss decreased steadily for all three variants, but BN fell furthest and fastest (1.48 to 0.45, versus 1.80 to 0.73 with no normalization and 1.76 to 0.62 with LRN). BN's validation loss showed one spike at epoch 6 (0.77 to 0.92) before recovering, while the unnormalized network's validation loss decreased smoothly but remained much higher. The clearest stability difference appeared on Tiny ImageNet: without normalization the network barely started learning (2.05% after epoch 1), and in the seed-42 LRN run training collapsed completely, with the loss fixed at ln(200) ≈ 5.298 and accuracy at chance (0.50%) for all ten epochs. BN trained normally in every run.

4.2 Data Augmentation

Generalization gap reduction: Yes, applying the PyTorch data augmentation pipeline reduces the generalization gap. Without augmentation, final training accuracy was 90.31% against 78.59% validation accuracy, a gap of $\Delta = 11.72$ points. With augmentation the final values were 84.64% and 81.65%, a gap of only 2.99 points, which is about 75% smaller. The unaugmented run shows classic overfitting: validation loss reached its minimum at epoch 11 and then rose while training loss kept falling.

Validation metric improvement: Yes, data augmentation led to an overall increase in final validation accuracy and Macro F1. Augmentation raised best validation accuracy from 81.18% to 82.83% and Macro F1 from 81.08% to 82.90% on CIFAR-10, and accuracy from 30.15% to 31.65% on Tiny ImageNet. These gains (+1.65 and +1.50 points) are within the ±0.88-point seed variation measured in Section 3.3, so augmentation's clearest benefit is the reduced generalization gap rather than peak accuracy. Random crops, horizontal flips and colour jitter preserve the label while changing position, orientation and colour, so the network never sees the same image twice. It cannot reduce the loss by memorizing specific pixel arrangements and is pushed toward features that stay the same under these changes, such as object shape and texture. Training accuracy is lower with augmentation because the training images are harder, which is the intended effect.

The improvement came at a cost. Epochs took 59.53 s with augmentation versus 26.58 s without. Training itself ran on the GPU, but the random transforms are applied in the CPU data loader, and with only two CPU cores on Colab the GPU spent much of each epoch waiting for augmented batches.

4.3 Tiny ImageNet

Generalization above random chance: Yes, AlexNet learns discriminative features on Tiny ImageNet significantly above the random guess baseline. AlexNet with BatchNorm and augmentation reached 31.65% validation accuracy (Macro F1 30.43%) after ten epochs, about 63 times the 0.5% chance level for 200 classes. Every model except the collapsed LRN run was well above chance.

Cross-dataset comparison:

The same model reached 82.83% on CIFAR-10 but 31.65% on Tiny ImageNet. Tiny ImageNet has 20 times as many classes, many of them fine-grained and visually similar, and only 500 training images per class versus 5,000 for CIFAR-10. Its 64 × 64 images are downsampled photographs of varied scenes with more clutter per image. With fewer examples per class and more classes to separate, the network has much less evidence for each decision boundary. Macro F1 sat a few points below accuracy on every Tiny ImageNet run, which indicates that some classes were rarely predicted correctly.

Overfitting trajectory:

Tiny ImageNet did not overfit within ten epochs. Validation loss decreased overall, apart from one rise at epoch 5, until it levelled off at about 2.90 in epochs 9 and 10, and validation accuracy stayed above training accuracy throughout because dropout and augmentation are active only during training. The model is under-trained rather than

overfitted. This contrasts with the unaugmented CIFAR-10 run, whose validation loss began rising after epoch 11. Tiny ImageNet also cost much more per epoch (176 s versus 27 s), mostly from loading 100,000 separate JPEG files each epoch.

Comparison of Model Variations

LeNet to Modernized LeNet: Replacing Tanh with ReLU and average pooling with overlapping max pooling raised CIFAR-10 accuracy from 55.23% to 66.65% and Tiny ImageNet accuracy from 18.95% to 26.64%, with only 0.11M more parameters. Tanh saturates for large inputs, where its gradient approaches zero and learning slows, while ReLU passes gradients unchanged for positive inputs. Max pooling keeps the strongest response in each window instead of averaging it with weaker neighbours.

LeNet to AlexNet: Moving to five convolutional stages with far more channels raised accuracy to 73.52% even without normalization, at the cost of about 20 times the parameters (6.6M) and 60 times the FLOPs (437M). Without normalization, however, this deeper network trained slowly on Tiny ImageNet and finished below the Modernized LeNet (18.03% versus 26.64%). Depth only paid off on the harder dataset once BN made it trainable.

AlexNet-64 versus the PyTorch Zoo AlexNet: The Zoo model has 57.0M parameters, about nine times as many, yet scored 65.02% on CIFAR-10 and 7.51% on Tiny ImageNet, far below AlexNet-64, having 79.93% and 30.15%. It was designed for 224 × 224 inputs: its 11 × 11, stride-4 first convolution and three pooling stages shrink a 64 × 64 image to a 1 × 1 feature map, which AdaptiveAvgPool2d then copies back up to 6 × 6. The 9,216 classifier inputs therefore contain only 256 distinct values, and most spatial information is lost before classification. It also needs fewer FLOPs (188M) despite its size, because its convolutions operate on tiny feature maps while most of its parameters sit in large fully connected layers that perform few operations per weight. AlexNet-64's first-layer changes (5 × 5 kernel, stride 2) keep a 3 × 3 map at the end of the feature extractor, which is why it performs far better at this resolution.

Run-to-Run Variability

Across seeds 42, 43 and 44, AlexNet + BN on CIFAR-10 reached 79.56 ± 0.88% best validation accuracy. Differences in the extended ablation table smaller than about twice this standard deviation (for example LRN's +1.62 points on CIFAR-10) are within run-to-run variation, while the BN gains of +6.41 and +12.12 points are well outside it. The LRN

collapse on Tiny ImageNet shows that some configurations are sensitive to initialization, which single-seed results can hide.